

ASAD – Jeu de roulette

Mouti Amir, N'dri Yao Anassa, Robin Steiger

1 : Introduction

Le but de ce travail pratique est de développer une application avec une architecture Data Centered. Cela veut dire une architecture de type « blackboard », où la gestion des données est centralisée sur un « tableau noir ». Les communications entre les différents processus se fait à travers ce blackboard central, sur lequel ils lisent et écrivent toutes les données.

L'application que nous allons développer avec cette architecture est un jeu de roulette multijoueur. Les joueurs jouent sur un tapis partagé et chaque partie de roulette a une fenêtre de temps dans laquelle les joueurs peuvent miser sur des cases. Chaque case ne peut accepter un pari d'un seul joueur à la fois et les joueurs peuvent modifier leurs paris pendant la fenêtre de temps imparti. Une fois le temps écoulé, le numéro gagnant est tiré et les gains et pertes sont calculées avant que la partie suivante commence.

Le code source peut être consulté dans le GitHub du projet :

<https://github.com/RobinSteiger/ASAD>

2 : Modélisation

Nous avons commencé par définir les **use cases** de notre système. Pour se donner une première idée de la structure de notre application, nous avons décidé de définir en premier les fonctions présentes.

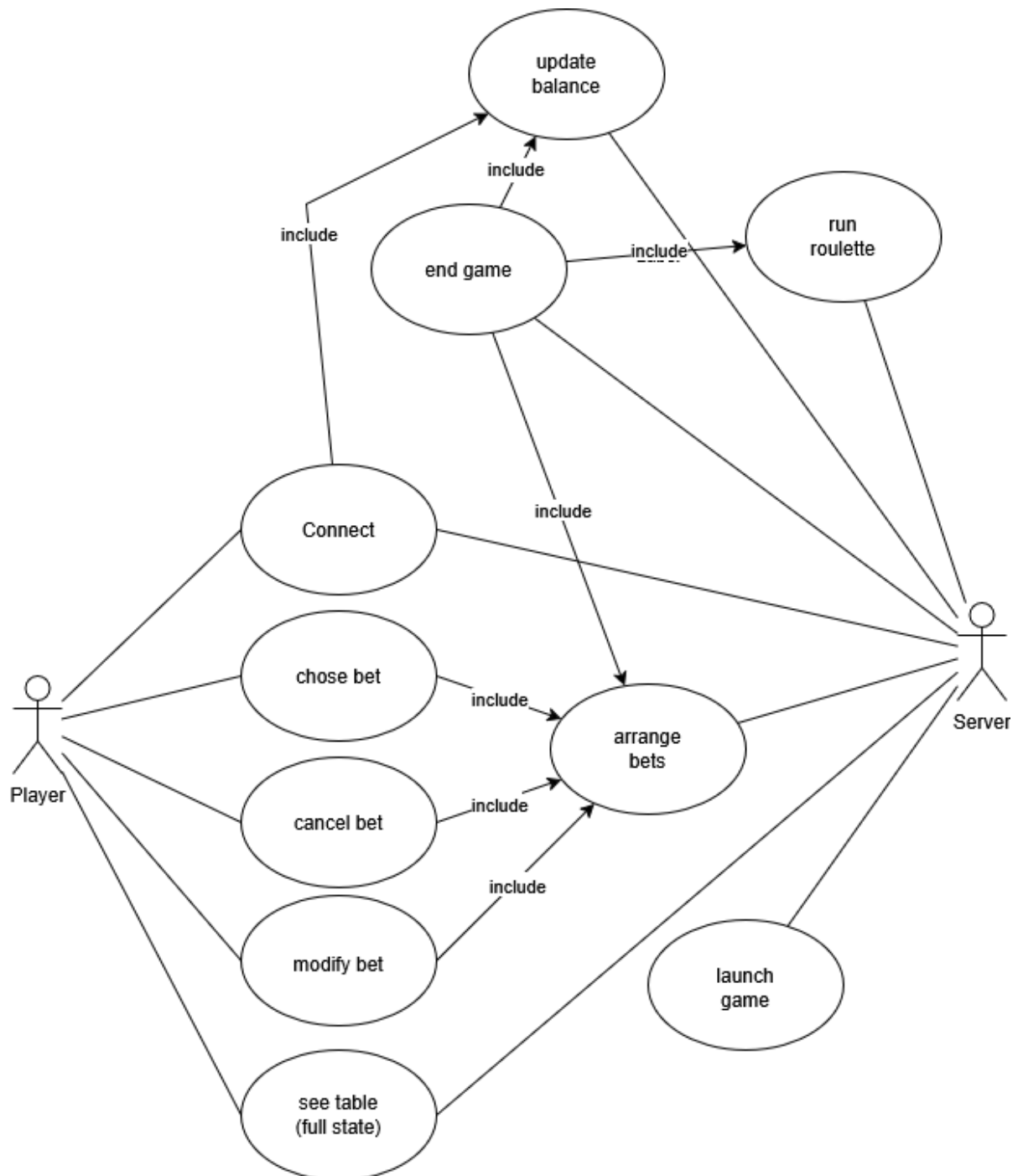


Figure 1 : Usecase Diagram

Nous avons identifié les fonctions présentes dans la Figure 1. Nous avons décidé de construire notre application avec une architecture client – serveur, car dans notre cas pour une architecture *data – centered*, il nous a semblé naturel que le blackboard central (qui stocke toutes les données) soit géré par un serveur qui l’initialise et fournit aux acteurs (dans ce cas les joueurs) les fonctions qui permettent de lire et écrire sur le « blackboard ».

Ensuite les diagrammes que nous avons définis avant de passer à la phase de conception sont le **Data Flow Diagram** et le **Requirements Diagram**. Nous avons décidé que ces diagrammes étaient les plus pertinents à établir à l’avance tandis que le reste des diagrammes peuvent changer au cours du développement de l’application.

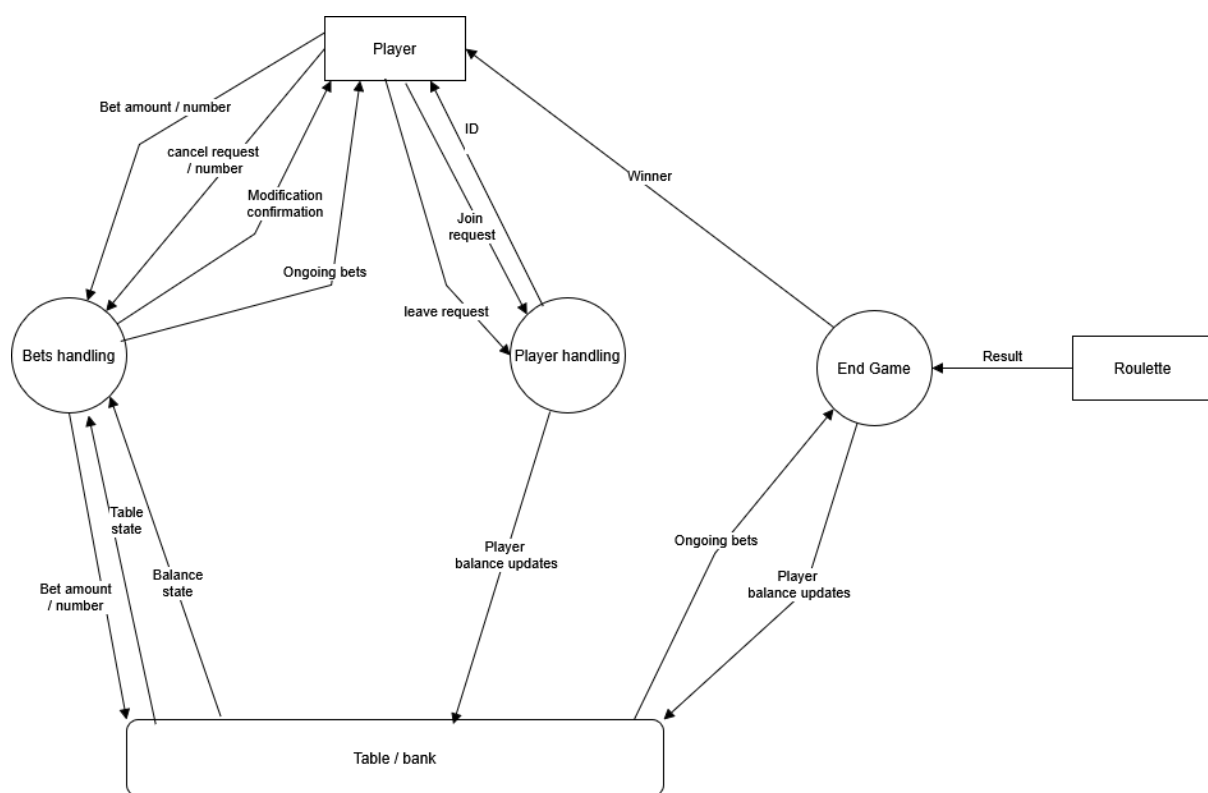


Figure 2 : Data Flow Diagram

Pour le **Data Flow Diagram** (figure 2), étant donné l’architecture *data-centered*, nous n’avons défini qu’un seul Data Store pour mettre l’importance sur le modèle « blackboard » où toutes les données sont stockées à un endroit central.

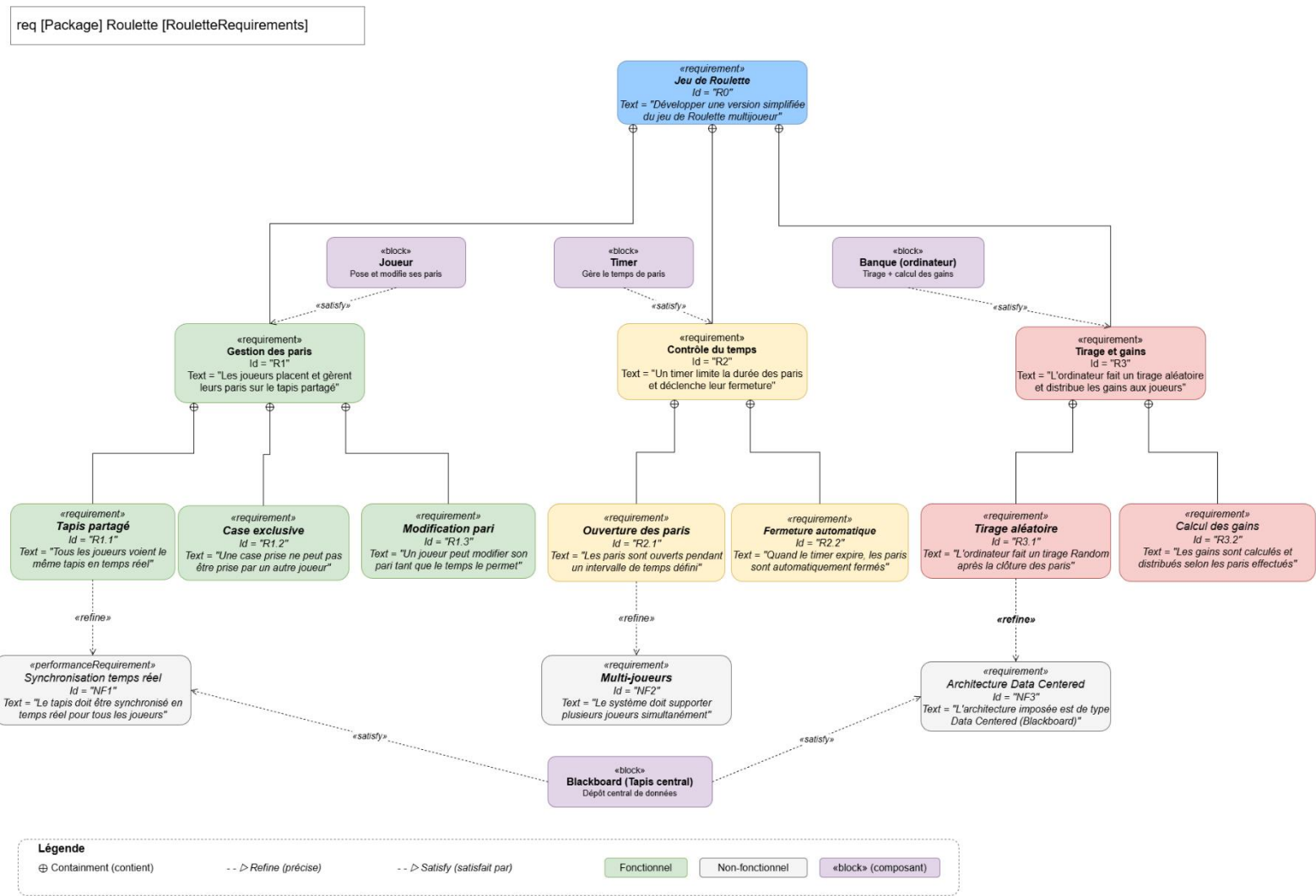


Figure 2 : Requirements Diagram

Dans le **Requirements Diagram** (figure 3), nous soulignons les exigences que nous avons identifiées pour notre application. Une fois ces trois diagrammes finalisés, nous avons commencé la phase d'implémentation. Selon nous, ces trois diagrammes nous donnent les détails les plus important sur l'architecture. Le reste des diagrammes ont été établis au cours du projet.

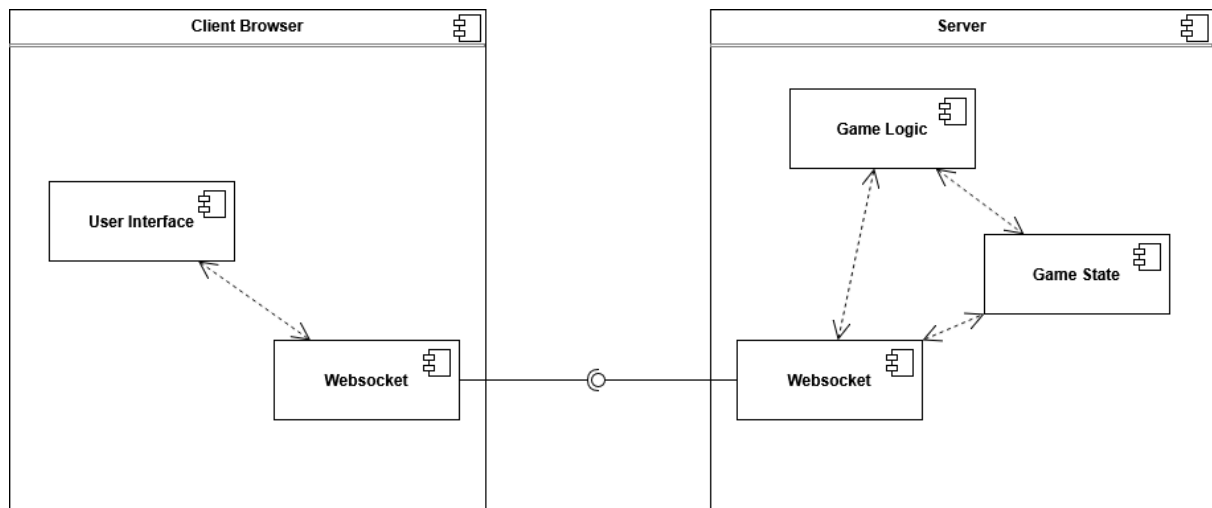


Figure 4 : Diagramme de composant

Pour le **Diagramme de Composants** (figure 4), nous avons une structure simple. Le client ne se charge que d’afficher les données reçues par le serveur et d’envoyer les actions que l’utilisateur souhaite exécuter. Du côté serveur, on modélise le blackboard par un composant « Game State » qui contient la liste des joueurs avec l’argent qu’ils ont à leur disposition pour faire des paris et la liste des paris en cours.

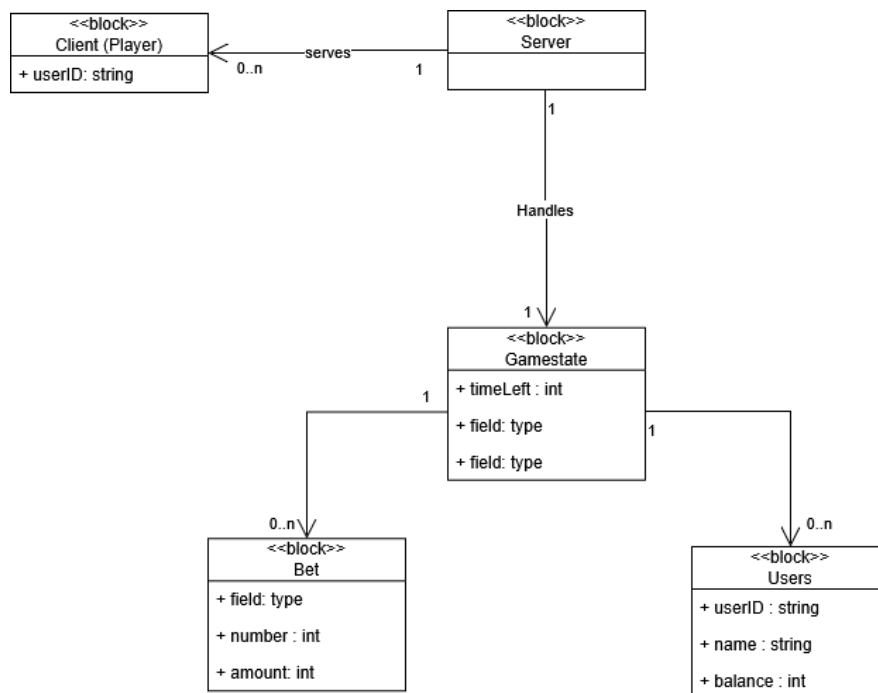


Figure 5 : Diagramme de Blocs

Dans le **Diagramme de Blocs** (Figure 5), on définit plus précisément les parties de l'implémentations. Pour ne pas trop encombrer le diagramme, tous les attributs ne sont pas indiqués (par exemple pour le client, il doit garder les informations sur le game state pour afficher l'interface). Nous avons aussi omis le composant Game Logic du diagramme de blocs car dans l'implémentation la logique du jeu sera implémentée par des fonctions du Game State et non pas un bloc séparé agissant sur le Game State.

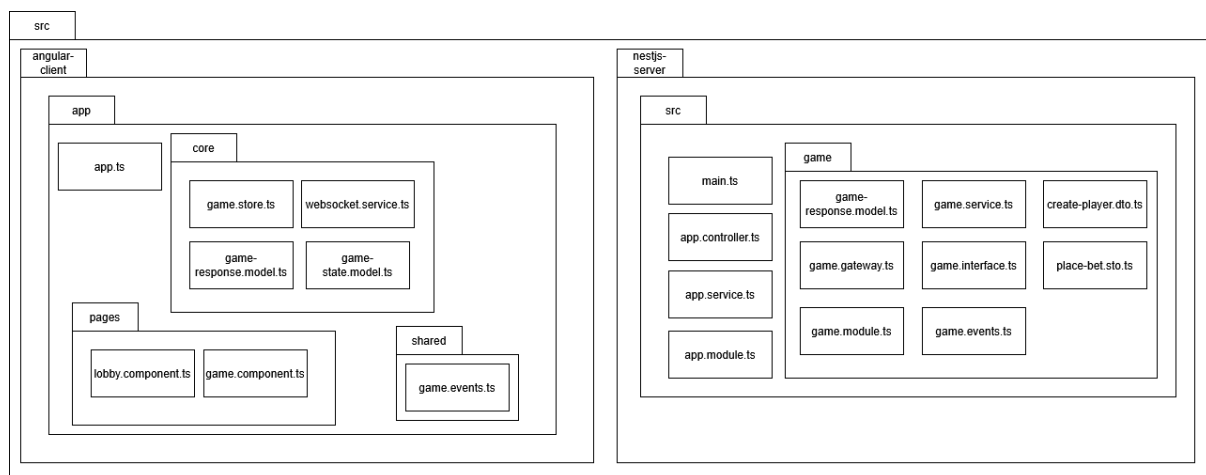


Figure 6 : Diagramme de Déploiement

Finalement le **Diagramme de Déploiement** (Figure 6) a été fait à la fin par rapport à l'état final de l'application. Nous n'avons pas ressenti qu'il était nécessaire de l'établir pour pouvoir implémenter l'application, nous ne l'avons donc fait qu'en dernier.

3 : Conception et Implémentation

3.1 Choix des technologies

- **Frontend : Angular 18+**
 - Utilisation des **Signals** pour une réactivité optimale de l'interface utilisateur sans surcharge de détection de changements.
 - **Tailwind CSS** pour un design "Dark Mode" immersif et responsive.
- **Backend : NestJS**
 - Choix de NestJS pour son architecture modulaire et son support natif de **TypeScript**, permettant un typage strict entre le serveur et le client.
 - **Socket.io** pour la communication bidirectionnelle en temps réel.

3.2 Conception

Le serveur orchestre un cycle de jeu automatisé divisé en trois phases distinctes :

1. **Phase de Mise (30s)** : Les utilisateurs peuvent placer des jetons sur le plateau.
2. **Phase de Tirage** : Fermeture des mises, génération du nombre aléatoire (RNG) et calcul des gains (Payout).
3. **Phase de Résultat (5s)** : Diffusion du résultat et affichage d'une popup de victoire/défaite synchronisée chez tous les joueurs.

3.3 Implémentation et problématique rencontrée

Lors de l'implémentation, plusieurs difficultés ont émergé de nos classes.

- **Problème de synchronisation** : Initialement, l'utilisation de JavaScript pur au backend créait des instabilités de typage avec le frontend Angular.
- **Solution** : Migration vers NestJS. Cela a permis de définir des interfaces communes (Modèles), garantissant que chaque message reçu par Angular est exactement au format attendu par ses Signaux.
- **Sécurité des sessions** : Implémentation d'un DestroyRef côté client et d'un contrôle de l'état (hasActiveBets) pour empêcher un joueur de quitter la table par erreur avec une mise en cours.

4 : Validation et Test

4.1 Tests et Validation

Afin de valider et de tester le fonctionnement de l'application, nous l'avons soumise à quelques tests simples.

- **Multijoueur** : Tests validés avec plusieurs instances de navigateurs. Le chronomètre et le numéro gagnant sont identiques pour tous les utilisateurs à la milliseconde près.
- **Expérience Utilisateur (UX)** : L'interface change d'état dynamiquement (bouton "Leave Table" désactivé pendant les mises, animations lors de l'apparition du résultat).

4.2 Améliorations futures

L'application finale, bien que complète, pourrait, dans le futur, bénéficier de plusieurs améliorations afin de pousser son développement plus loin.

- **Persistance** : Ajout d'une base de données (PostgreSQL ou MongoDB) pour sauvegarder les soldes des joueurs après déconnexion.
- **Historique** : Affichage des 10 derniers numéros sortis sous forme de graphique.
- **Animations** : Intégration d'une roue de roulette animée en CSS/Canvas pour remplacer l'affichage textuel du résultat.

5 : Conclusion

Les objectifs définis au début du projet ont pu être atteints. L'application finale permet effectivement à plusieurs joueurs de parier et participer simultanément à un jeu de roulette.

Les divers diagrammes définis au cours du développement nous ont permis d'identifier plus facilement les différents composants, cas d'utilisation et problématique qui composent l'application.

Dans l'ensemble, l'aboutissement de ce travail est satisfaisant et nous a permis d'appréhender une approche de développement originale.